

Task 1 - Data Extraction

Del documento Business Case A&A

Data Extraction:

You will be provided with a set of tables and corresponding columns. Your first task is to develop an efficient SQL query that extracts the required data and establishes a pipeline to produce a dataset similar to the one attached to this case. The query should dynamically calculate the date range for the previous week based on the current date ({{ds}}), and be designed to refresh on a weekly basis. The final output should be saved to a table within the AA_tables schema.

Basado en eso:

```
/* -----  
-----  
  
Supuesto clave:  
  
{{ds}} : parametro de entrada que representa la fecha de ejecución en  
formato YYYY-MM-DD.  
  
-----  
----- */  
  
WITH runtime_dates AS ( -- Creamos una CTE que calcula el inicio y fin de la  
semana  
  
SELECT  
  
-- date_trunc solito lo que hace es truncar ds al inicio de la semana  
  
-- Al restarle INTERVAL '1 week' obtenemos el inicio de la semana pasada  
  
date_trunc('week', {{ ds }}::timestamp without time zone) - INTERVAL '1  
week' AS wk_start,  
  
-- Lo mismo pero solo restamos 1 segundo para obtener el fin de la semana  
pasada  
  
date_trunc('week', {{ ds }}::timestamp without time zone) - INTERVAL '1  
second' AS wk_end
```

```

),

base_dispatch AS (

SELECT

dmjm.jobuuid AS workflow_uuid, -- para hacer el join con
tmp.lea_trips_scope_atd_consolidation_v2

dmjm.cityid AS city_id, -- para hacer el join con dwh.dim_city

dmjm.datestr::timestamp AS order_datetime_utc, -- para filtrar por fecha

dmjm.pickupdistance / 1000 AS pickup_distance, -- Unica tabla donde se puede
para obtener la distancia de recogida en km

dmjm.traveldistance / 1000 AS dropoff_distance, -- para obtener la distancia
de entrega en km

dmjm.isfinalplan -- asuemo aquí que solo queremos entregas finales

FROM delivery_matching.eats_dispatch_metrics_job_message dmjm

JOIN runtime_dates rd -- rd solo tiene una fila entonces este join funciona
directemnent como un filtro

ON dmjm.datestr::timestamp BETWEEN rd.wk_start AND rd.wk_end

WHERE dmjm.isfinalplan = TRUE -- asuemo aquí que solo queremos entregas
finales

),

trip_scope AS ( -- obtenemos cada viaje con sus detalles

SELECT

sc.delivery_trip_uuid,

sc.workflow_uuid, -- para hacer el join con base_dispatch

-- Estos campos ya tienen el nombre para la tabla de appendiz 2

sc.driver_uuid,

```

```

sc.courier_flow,

sc.restaurant_offered_timestamp_utc,

sc.order_final_state_timestamp_local,

sc.eater_request_timestamp_local,

sc.geo_archetype,

sc.merchant_surface,

FROM tmp.lea_trips_scope_atd_consolidation_v2 sc

),

city_enrichment AS ( -- Enriquecemos con la dimension ciudad

SELECT

dc.city_id, -- para hacer el join con base_dispatch

-- Estos campos ya tienen el nombre para la tabla de appendiz 2

dc.country_name,

csr.region,

csr.territory

FROM dwh.dim_city dc

LEFT JOIN kirby_external_data.cities_strategy_region csr -- La necesitamos
para region y territory

ON csr.city_id = dc.city_id

)

SELECT

ce.territory,

ce.country_name,

ts.workflow_uuid,

```

```

ts.driver_uuid,

sc.delivery_trip_uuid,

ts.courier_flow,

ts.restaurant_offered_timestamp_utc,

ts.order_final_state_timestamp_local,

ts.eater_request_timestamp_local,

ts.geo_archetype,

ts.merchant_surface,

bd.pickup_distance,

bd.dropoff_distance,

EXTRACT(EPOCH FROM -- EPOCH extrae la diferencia en segundos

(ts.order_final_state_timestamp_local -
ts.restaurant_offered_timestamp_utc)) / 60 AS atd

FROM base_dispatch bd

JOIN trip_scope ts USING (workflow_uuid)

JOIN city_enrichment ce USING (city_id)

WHERE ce.country_name = 'Mexico';

```

- I believe this query already fulfills the purpose. However, the doubt remains of whether it is okay to assume that jobuuids in delivery_matching.eats_dispatch_metrics.job_message is the same as workflow_uuid, I couldn't see how else to manage that join.

**Workflow discussion+*

- This query is already set to be run weekly through an orchestrator that solely pulls the data for the last week.
1. Trigger is assumed to be scheduled once a week, hence the query will only pull (thanks to the CTE runtime dates) the data from 00:00:00 Monday at the start of the week up until

23:59:59 Sunday. Which would mean the next run would cover the missing second iteratively.

2. Results would be delta inserted into AA_with_ATD
3. To ensure appropriate data lineage, an Archive job in the pipeline should keep track of:
 1. valid_from : the date when the record became active in AA_with_ATD
 2. valid_to : the date when the record was last replaced or made inactive
 3. is_current : boolean for active records
4. We would only stage current records and prepare them for transformation and view creation, but not current records would be saved for proper change traceability.
5. Assuming an indexing strategy is in place for dates and certain uuids, then no work would be needed there. In case there are no indexes in place, a date index would be at least required to reduce latency and improve query performance in the later stage of the ETL.